

SonarQube + Semgrep CI/CD Trial — Findings

TL;DR

Both tools work well together in GitHub Actions. SonarCloud is the right choice over self-hosted SonarQube for any team that doesn't have a compliance reason to self-host. Semgrep is free and tokenless but needs explicit rulepacks and correct job permissions to surface results in GitHub Security.

SonarQube vs SonarCloud

Use SonarCloud unless you have a reason not to.

	SonarQube (self-hosted)	SonarCloud (hosted)
Hosting	You run it	Sonarsource runs it
Cost	Free (Community Edition)	Free for public repos
PR decoration	Developer Edition+ (paid)	Included
Branch analysis	Developer Edition+ (paid)	Included
Setup effort	High — run a server	Low — OAuth with GitHub
CI integration	Needs scanner + server config	Actions action + token

Key limitation of Community Edition: It only scans the main branch. No PR decoration, no branch comparison. You need Developer Edition (paid) or SonarCloud to get inline PR comments and per-branch analysis.

SonarCloud Setup — What Actually Works

Step 1 — Sign up

- sonarcloud.io → Log in with GitHub
- It will immediately run **Automatic Analysis** on your repo — this is not what you want for CI/CD

Step 2 — Disable Automatic Analysis

This is non-obvious and easy to miss. Without doing this, SonarCloud and your Actions workflow will conflict and double-scan.

- Project → **Administration** → **Analysis Method** → turn off **Automatic Analysis**

Step 3 — Generate a token

- Avatar → **My Account** → **Security** → Generate token
- Add to GitHub repo: **Settings** → **Secrets and variables** → **Actions** → `SONAR_TOKEN`

Step 4 — Get your project key and org

- Project → **Information** (bottom left sidebar)
- You need both `Project Key` and `Organization Key` for `sonar-project.properties`

sonar-project.properties (required at repo root)

```
sonar.projectKey=your-org_your-repo
sonar.organization=your-org
sonar.projectName=Your Project
sonar.projectVersion=1.0
```

```
sonar.sources=.
sonar.exclusions=**/.github/**
sonar.php.version=8.0
```

Change `sonar.php.version` to match your stack. For Node.js projects remove it entirely.

GitHub Actions job

```
sonarcloud:
  name: SonarCloud Scan
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0 # required – SonarCloud needs full history for blame/new-code
    - uses: SonarSource/sonarcloud-github-action@master
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

fetch-depth: 0 is required. Without it, SonarCloud can't calculate new code or blame data correctly.

Exporting issues as an artifact

Add these steps after the scan to download findings as JSON for passing to AI:

```
- name: Wait for SonarCloud background task to complete
  run: |
    for i in $(seq 1 24); do
      STATUS=$(curl -s -u "${ secrets.SONAR_TOKEN }": \
        "https://sonarcloud.io/api/ce/activity?component=YOUR_PROJECT_KEY&ps=1" \
        | python3 -c "import sys,json; tasks=json.load(sys.stdin).get('tasks',[]);
print(tasks[0]['status'] if tasks else 'PENDING')")
      echo "Attempt $i – task status: $STATUS"
      if [ "$STATUS" = "SUCCESS" ] || [ "$STATUS" = "FAILED" ] || [ "$STATUS" = "CANCELLED"
]; then
        break
      fi
      sleep 10
    done

- name: Export SonarCloud issues
  run: |
    curl -s \
      -u "${ secrets.SONAR_TOKEN }": \
      "https://sonarcloud.io/api/issues/search?
projectKeys=YOUR_PROJECT_KEY&resolved=false&ps=500" \
      -o sonarcloud-issues.json

- name: Save SonarCloud issues as artifact
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: sonarcloud-issues
    path: sonarcloud-issues.json
```

The polling step is required. SonarCloud processes analysis async — without it the export runs before results are ready and returns stale data from the previous scan.

Semgrep Setup — What Actually Works

Free plan limitation

Semgrep's free plan does not give you an App Token. The token is required to use `--config=auto` and to report findings to the Semgrep dashboard.

Workaround: Use explicit public rulepacks instead. No token needed, results still upload to GitHub Security via SARIF.

`--config=auto` without a token = no rules

This is the key gotcha. `--config=auto` silently fetches a near-empty ruleset when unauthenticated — it will find nothing. Use explicit configs instead:

```
--config=p/php          # PHP language rules
--config=p/wordpress    # WordPress-specific rules
--config=p/security-audit
--config=p/owasp-top-ten
--config=p/secrets
```

For Node.js projects swap `p/php` and `p/wordpress` for `p/javascript` and `p/nodejs`.

`--error` blocks the SARIF upload

Using `--error` exits non-zero when findings are found, killing the job before the upload step runs. Use `|| true`:

```
semgrep scan ... || true
```

Job needs `security-events: write` permission

Without this the `upload-sarif` step silently fails and nothing appears in GitHub Security.

```
semgrep:
  name: Semgrep Scan
  runs-on: ubuntu-latest
  permissions:
    contents: read
    security-events: write # required for SARIF upload to GitHub Security
    actions: read
```

GitHub's generated Semgrep workflow requires a token

If you use GitHub's built-in **Security → Code scanning → Add tool → Semgrep** flow, it generates a workflow using `returntocorp/semgrep-action` which requires `SEMGREP_APP_TOKEN`. This won't work on the free plan. Use the manual approach below instead.

Full working Semgrep job (no token)

```
semgrep:
  name: Semgrep Scan
  runs-on: ubuntu-latest
  permissions:
    contents: read
```

```

security-events: write
actions: read
container:
  image: semgrep/semgrep
steps:
  - uses: actions/checkout@v4

  - name: Run Semgrep
    run: |
      semgrep scan \
        --config=p/php \
        --config=p/wordpress \
        --config=p/security-audit \
        --config=p/owasp-top-ten \
        --config=p/secrets \
        --sarif \
        --output=semgrep.sarif || true

  - name: Upload SARIF to GitHub Security
    uses: github/codeql-action/upload-sarif@v3
    if: always()
    with:
      sarif_file: semgrep.sarif
      category: semgrep

  - name: Save SARIF as downloadable artifact
    uses: actions/upload-artifact@v4
    if: always()
    with:
      name: semgrep-sarif
      path: semgrep.sarif

```

Coverage Matrix — WordPress/PHP (verified against bad-plugin.php)

Tested against an intentionally insecure WordPress plugin. **SonarCloud: 39 findings. Semgrep: 10 findings.**

Issue	Line	SonarCloud	Semgrep	Notes
Hardcoded API key	10	BLOCKER	—	Semgrep p/secrets missed it
Hardcoded password	8	—	—	Neither caught it
Hardcoded GitHub token	11	—	—	Neither caught it
SQL injection (search_posts)	24	BLOCKER	MEDIUM	Both caught it
SQL injection (get_user_data)	16	—	—	Neither caught it — no direct user input flow
XSS \$_GET['name']	30	BLOCKER	HIGH	Both caught it
XSS \$_REQUEST['message']	35	BLOCKER	HIGH	Both caught it
XSS post_content echoed	112	—	HIGH	Semgrep only
CSRF — no nonce check	40	—	—	Neither caught it
Local file inclusion	52	BLOCKER	—	SonarCloud only

Remote file inclusion	58	BLOCKER	—	SonarCloud only
Arbitrary file deletion	64	BLOCKER	HIGH	Both caught it (as path traversal)
Command injection system()	70	BLOCKER	HIGH x2	Both caught it
Insecure file upload	74	—	—	Neither caught it
MD5 password hashing	85	CRITICAL	—	SonarCloud only
Plain text password in user meta	89	—	—	Neither caught it
Privilege escalation (no cap check)	94	—	—	Neither caught it
Unauthenticated AJAX handler	101	—	—	Neither caught it
PHP unserialize on user input	124	BLOCKER	—	SonarCloud only
Path traversal (file read)	119	BLOCKER	HIGH x2	Both caught it
Open redirect	132	BLOCKER	—	SonarCloud only
phpinfo() exposed	138	—	HIGH	Semgrep only
Hardcoded admin password on activate	143	—	—	Neither caught it
var_dump in footer	152	BLOCKER	HIGH	Both caught it (as XSS/reflection)

Gaps — neither tool caught

- Hardcoded password and GitHub token (line 8, 11)
- CSRF (no nonce verification) — logic-level, tools can't easily detect without framework-specific rules
- Insecure file upload (no validation) — requires data flow analysis across WordPress functions
- Plain text password stored in user meta
- Privilege escalation / missing capability checks — requires WordPress context awareness
- Unauthenticated AJAX handler (`wp_ajax_nopriv_`) — WordPress-specific pattern
- Hardcoded admin credential reset on activation

Why the gaps matter

Several of the missed issues (CSRF, privilege escalation, unauthenticated AJAX) are the **most common real-world WordPress vulnerabilities** according to WPScan data. SAST tools struggle with them because they require understanding WordPress hook and capability patterns. For real WordPress projects consider adding **WPScan** or a dedicated WordPress security scanner alongside these two tools.

SonarCloud code smell noise

SonarCloud flagged 20+ MINOR issues for function naming convention (`php:S100` — snake_case vs camelCase). These are valid but low priority — worth suppressing with a custom Quality Profile if they're drowning out the real findings.

Quality Gate (blocking merges on bad code)

SonarCloud enforces a Quality Gate — a pass/fail threshold that can block PRs from merging.

- Default gate: fails if new code has any bugs, vulnerabilities, or security hotspots
- Configure at: Project → **Administration** → **Quality Gate**
- To block PRs: GitHub repo **Settings** → **Branches** → **Branch protection rules** → add the SonarCloud check as required

PR Decoration (inline comments on PRs)

- **SonarCloud:** Works out of the box on the free plan. Findings appear as inline comments on the PR diff.
- **Semgrep:** Findings appear in **GitHub Security** → **Code scanning** tab. Inline PR comments require a paid Semgrep account.

Getting Findings into AI for Fixes

Both tools export their findings as downloadable artifacts on every Actions run:

- **Actions** → **your run** → **Artifacts** → `sonarcloud-issues` (JSON) and `semgrep-sarif` (SARIF)

Paste either file into Claude with: *"Here are the findings from our SAST scan. Please fix all issues in the codebase."* Claude understands both formats natively and can work through all fixes in one pass.

Gotchas Summary

Gotcha	Fix
SonarCloud runs Automatic Analysis by default	Disable it: Administration → Analysis Method
SonarCloud export returns stale data	Add polling step — waits for background task to finish
<code>--config=auto</code> finds nothing without a token	Use explicit rulepacks (p/php, p/wordpress, etc.)
Semgrep SARIF upload silently fails	Add <code>security-events: write</code> to job permissions
<code>--error</code> kills job before SARIF upload	Use <code> true</code> on the scan command
GitHub's generated Semgrep workflow needs a paid token	Write your own job using the <code>semgrep/semgrep</code> container
SonarCloud <code>fetch-depth: 0</code> missing	Always set it — SonarCloud breaks without full git history
GitHub UI adds conflicting workflow files	Pull before pushing if you've edited workflows via GitHub UI
SonarCloud MINOR noise from naming conventions	Suppress <code>php:S100</code> in a custom Quality Profile

Recommended Rulepacks by Stack

Stack	Semgrep configs
WordPress / PHP	<code>p/php p/wordpress p/security-audit p/owasp-top-ten p/secrets</code>
Node.js	<code>p/javascript p/nodejs p/security-audit p/owasp-top-ten p/secrets</code>
TypeScript	<code>p/typescript p/nodejs p/security-audit p/owasp-top-ten p/secrets</code>
Python	<code>p/python p/security-audit p/owasp-top-ten p/secrets</code>